

access to specific areas of the system. This helps to ensure that only authorised personnel are able to make changes to the system.

Running CronJobs in parallel with EKS

By default, Kubernetes CronJobs runs jobs sequentially. To run CronJobs in parallel, you can set the `.spec.concurrencyPolicy` field in the CronJob spec to 'Allow'.

```
spec:
  schedule: "*/5 * * * *"
  concurrencyPolicy: "Allow"
```

To run the same job in parallel, type the following command:

```
apiVersion: batch/v1beta1
kind: Cron Job
metadata:
  name: my-cronjob
spec:
  schedule: "*/5
  parallelism: 2
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: my-container
              image: my-docker-image
              command: ["my-command"]
          restartPolicy: OnFailure
```

Running multiple jobs concurrently or in parallel can increase the load on the cluster and cause performance issues. It's important to select this workload after due diligence on the impact and by provisioning adequate resources.

Running non-container jobs from Kubernetes CronJob

This is very simple and involves using a Kubernetes pod template to run a command or script that can execute the external job.

- Create a pod template that runs the command or script that can execute the external job.
- This pod template runs a container with `alpine:latest` image and runs the `external-job-command` command.
- Create a CronJob that uses the pod template you just created. For example:

```
kind: Cron Job
metadata:
  name: my-cronjob
```

```
spec:
  schedule: "0 1 * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            name: my-container
            image: alpine:latest
            command: ["/bin/sh", "external-job-command"]
```

This CronJob schedules the pod to run the `external-job-command` command.

On-demand execution and forced restarts

To manually trigger a CronJob, use the `kubectl create job` command followed by the name of the CronJob. The Kubernetes API also provides a create endpoint for the `batch/v1beta1/cronjobs/{name}/trigger` resource to trigger a CronJob using an API call.

When a CronJob is manually triggered or restarted, a new job object is created with a unique name and a new pod is scheduled to run the workload. The old job object and pod are terminated, and any resources used by the old pod are released.

To configure a CronJob to automatically restart when it fails or completes, set the `spec.concurrencyPolicy` field to 'Forbid' (the default), 'Allow', or 'Replace'.

The *Forbid* policy prevents concurrent executions of the same CronJob. *Allow* allows concurrent executions, and *Replace* replaces the old job with a new one, if the old job is still running when the next scheduled execution time arrives.

Additionally, you can configure the `spec.failedJobsHistoryLimit` and `spec.successfulJobsHistoryLimit` fields to control the number of failed and successful job objects that are retained by the CronJob controller. This helps with troubleshooting and audit logging.

Preventing Kubernetes CronJobs from running on holidays

- To prevent jobs from running on holidays, you can use external holiday calendars or APIs to generate a list of holidays and exclude them from the schedule. You can also use Kubernetes Jobs to trigger CronJobs based on a file or job trigger (refer Figures 1, 2 and 3).
- Create a ConfigMap that contains a list of holidays in the format YYYY-MM-DD.
- Create a script that checks if the current date is a holiday, and exits with an error code if it is.
Mount the ConfigMap as a file in the pod that runs the CronJob.

In the above example, CronJob runs a container that executes a Bash script `/bin/holiday-check.sh` before running